FestiBed – Gestion de la concurrence et passage à l'échelle

Projet intégrateur M1 MIAGE – Groupe 3 – 23/10/2026 – Oracle · Spring Boot réactif (WebFlux) · Kotlin · JPA

En une phrase : pendant que le festivalier remplit son panier, rien n'est bloqué ; quand il valide, une seule transaction très courte vérifie et réserve ensemble les Pass et les nuitées, et c'est la base de données qui décide qui obtient la dernière place.

PARTIE 1 – POUR TOUT LE MONDE

1. Le problème

Les Pass (quota par festival) et les nuits d'hébergement sont **limités**. Si Alice et Bruno voient « 1 chambre disponible » et valident à la même seconde, la chambre risque d'être **vendue deux fois**. Objectif : **aucune survente**, et rester **rapide même avec des milliers de personnes**.

2. Notre stratégie en trois idées

1 Le panier ne bloque rien
Il reste dans le navigateur (15 min max). Personne n'est bloqué par un panier abandonné.

2 Tout se joue à la validation
Une seule opération réserve Pass et nuitées ensemble : tout réussit, ou rien n'est réservé.

3 La base décide qui gagne
Une nuitée n'est réservée que si elle est encore libre au moment précis de l'écriture.

Comme un siège d'avion : tout le monde peut regarder le plan, mais au paiement un seul obtient le siège 12A ; l'autre reçoit « siège déjà pris ».

3. Deux personnes, la dernière chambre

Temps	Alice	Bruno
1	Valide : la base réserve la chambre et la **verrouille**.	
2		Valide un instant après : il **attend** quelques millisecondes.
3	**Réservation confirmée** (Pass + bon d'hébergement).	
4		La chambre n'est plus libre : **rien n'est réservé**, message « plus disponible, choisissez-en un autre ».

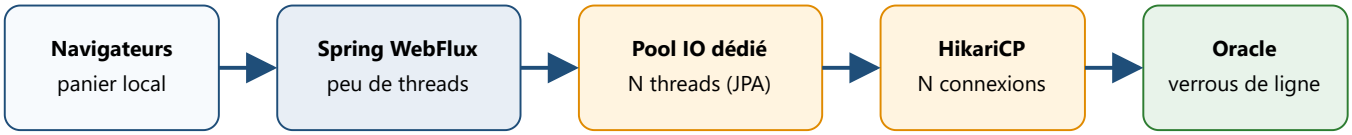
4. Pourquoi cela tient sous forte charge

- **Opérations très courtes** : la base n'est sollicitée que quelques millisecondes, à la validation.
- **On ne bloque que ce qui est disputé** : deux chambres différentes se réservent en parallèle.
- **Serveur réactif** : peu de threads servent des milliers de visiteurs ; le serveur ne reste jamais inactif à attendre.
- **Base protégée** : un nombre fixe d'accès simultanés à Oracle (des « guichets ») ; au-delà, on fait la queue dans l'application.

Garanties : jamais deux ventes de la même nuit ; jamais de Pass sans hébergement ; une panne pendant la validation n'enregistre rien ; le panier ne garantit pas la place, seule la validation la garantit.

PARTIE 2 – POUR L'ÉQUIPE TECHNIQUE

5. Architecture réactive



La file d'attente se forme dans l'application, jamais dans Oracle.

> **Point clé :** JPA est **bloquant** et sa transaction est **liée à un thread**. La validation est donc une fonction Kotlin **normale (pas `suspend`)** annotée `@Transactional`, lancée par `withContext` sur un pool IO de même taille que HikariCP.

6. Vérifier et réserver en une seule instruction

```
UPDATE Nuitee SET idSelection = :sel
 WHERE idEtablissement = :e AND numeroHebergement = :h
   AND dateNuitee IN (:nuits) AND idSelection IS NULL;         -- lignes < nuits → échec
UPDATE Festival SET nbPassVendus = nbPassVendus + :n          -- en DERNIER (ligne la plus disputée)
 WHERE idFestival = :f AND nbPassVendus + :n <= quotaPass;      -- 0 ligne → échec
```

Jamais « `SELECT` puis `UPDATE` » : la condition et l'écriture sont atomiques. Si une autre transaction tient le verrou, Oracle attend son COMMIT puis réévalue la condition : 0 ligne modifiée.

7. Code Spring réactif + Kotlin

```
// Contrôleur WebFlux : n'occupe jamais un thread réseau
suspend fun valider(panier: PanierDto, user: String): ResponseEntity<ReservationDto> {
    repeat(3) { essai ->
        try {
            val resa = withContext(jpaDispatcher) { validationService.valider(panier, user) }
            documentService.genererPdfAsync(resa)                 // après le COMMIT
            return ResponseEntity.ok(resa)
        } catch (e: IndisponibleException) { return ResponseEntity.status(409).build() }  // pas de
réessai
          catch (e: CannotAcquireLockException) { delay(50L * (essai + 1)) }               // réessai
    }
    return ResponseEntity.status(503).build()
}

// Service : transaction courte, tout ou rien
@Transactional(isolation = Isolation.READ_COMMITTED, timeout = 5)
fun valider(panier: PanierDto, user: String): ReservationDto {             // PAS suspend
    val resa = reservationRepo.save(Reservation(festivalier(user)))
    for (s in panier.selections.sortedBy { it.idFestival }) {              // ordre fixe
        val sel = selectionRepo.save(Selection(resa, s))
        for (h in s.hebergements.sortedBy { it.cle })
            if (nuiteeRepo.occuper(sel, h, h.nuits) < h.nuits.size) throw IndisponibleException()
        if (festivalRepo.reserverPass(s.idFestival, s.nbPass) == 0) throw IndisponibleException()
        passRepo.saveAll(Pass.generer(sel, s.nbPass))
    }
    return resa.toDto()                          // exception → ROLLBACK ; sinon → COMMIT
}
```

8. Règles de montée en charge et choix de l'approche

Règle	Effet
Transaction courte	PDF, emails, paiement après le COMMIT : verrous tenus quelques ms.
Point chaud en dernier	La ligne `Festival`, partagée par tous les acheteurs, est verrouillée le moins longtemps.
Ordre de verrouillage fixe	Festivals par id, nuitées par (établissement, hébergement, date) : pas d'interblocage.
Pool borné + timeout 5 s	N threads IO = N connexions : Oracle n'est jamais saturé.
Réessai ciblé	Seulement sur verrou / interblocage, jamais sur « plus de place ».

Approche	Survente ?	Sous forte charge
SELECT puis UPDATE	**Oui**	Deux clients lisent « libre » en même temps.
SELECT … FOR UPDATE	Non	Verrou tenu plus longtemps (deux allers-retours).
`@Version` (optimiste)	Non	Échecs en série sur un festival populaire alors qu'il reste des places.
UPDATE conditionnel (choix)	**Non**	**Un aller-retour, verrou minimal, échec seulement s'il ne reste rien.**

9. Preuves

Test concurrent (JUnit + coroutines) : N validations simultanées sur la dernière place → une seule réussit. **Deux sessions SQL** : la seconde attend le verrou puis modifie 0 ligne. **Test de charge** (Gatling/JMeter) : temps de réponse et débit mesurés, zéro survente en base.